Flutter Course
Dart Basics

Ruaa Abdullah

Course Road Map

- introduction
- **Dart programming basics**
- **Object oriented programming**
- **Flutter Fundamentals**
- **Flutter Advanced : BMI calculator app**
- **Todo app – Local data base SQFlite**
- **News APP-Apis and network**
- **Social APP / E- Commerce - Firebase**

Flutter Introduction

- Flutter is an open-source UI software development kit (SDK) created by Google. It is used to develop applications for mobile, web, and desktop from a single codebase. Flutter allows developers to write code once and deploy it across different platforms without having to rewrite the entire application for each platform.

- Flutter uses the Dart programming language, which was also developed by Google in 2017, as its primary language. Dart is a modern, object-oriented language with features such as strong typing, just-in-time (JIT) compilation (in runtime) (Hot reload), and ahead-of-time (AOT) compilation (before runtime) (Hot Restart).

- Flutter provides a rich set of pre-built widgets that developers can use to create beautiful and highly customizable user interfaces. These widgets are designed to look and feel native on each platform, providing a consistent user experience across different devices.

- Overall, Flutter has gained popularity among developers for its fast development cycle, expressive and flexible UI framework, and ability to target multiple platforms with a single codebase.

C2 General

Different between cross platform and native platform

- Native: Developing for a native platform involves creating applications specifically for a single platform, such as iOS or Android, using the respective platform-specific programming languages, APIs, and development tools (e.g., Swift or Objective-C for iOS, Java or Kotlin for Android).

- Cross-platform development involves creating applications that can run on multiple platforms (e.g., iOS, Android, and sometimes others like web and desktop) using a single codebase. Cross-platform frameworks and tools, such as Flutter, React Native, Xamarin, or Ionic, enable developers to write code once and deploy it across different platforms.

C2 General

How flutter works

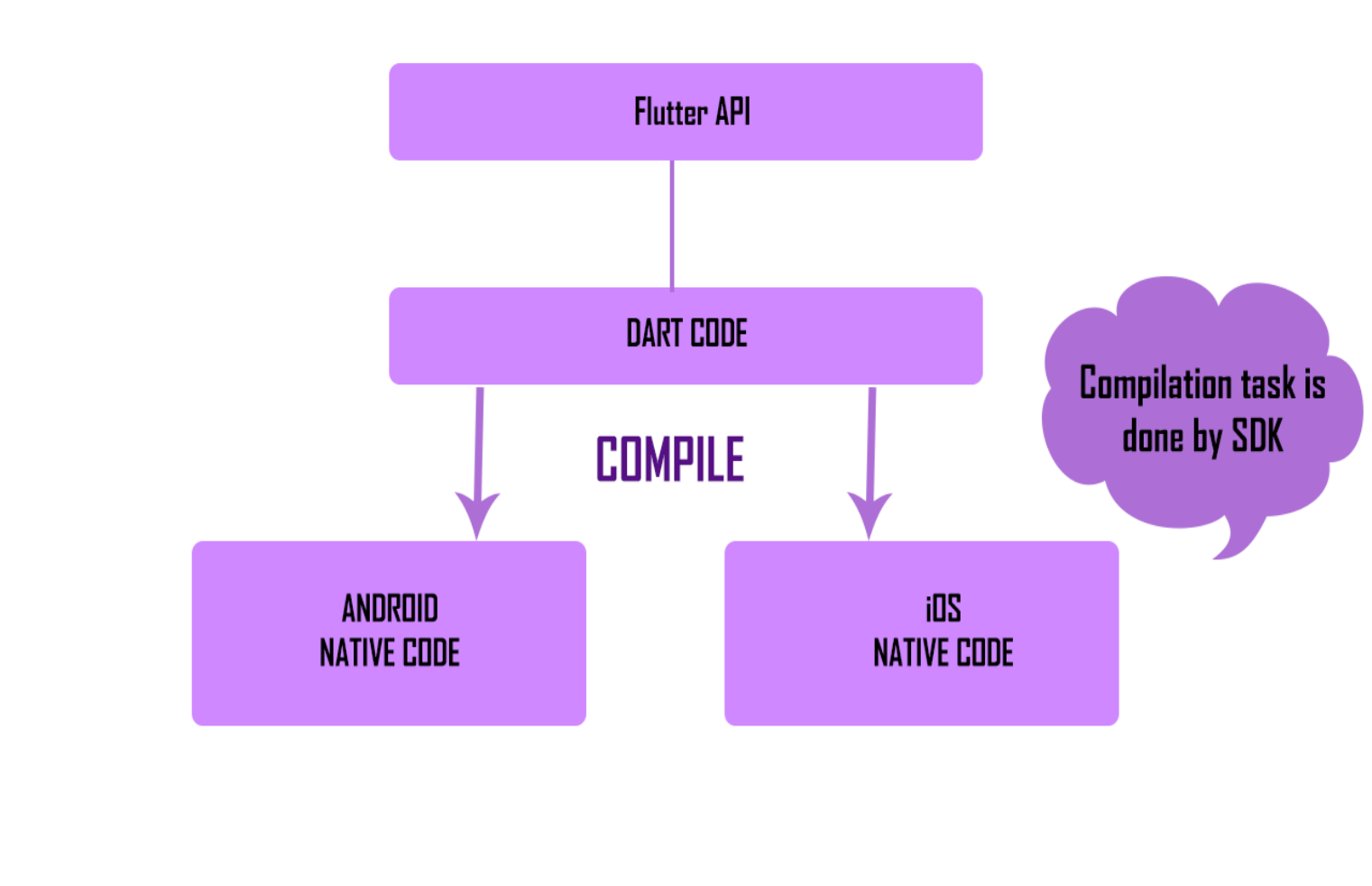
- Flutter works by providing a framework for building native interfaces for iOS, Android, and other platforms using a single codebase.

ANDROID: JAVA/KOTLIN

IOS:SWIFT

WEB:HTML-CSS_JAVASCRIPT

DESKTOP: C++



Setup environment

- DOWNLOAD : Android Studio Website
- Windows :How to install Flutter on windows
- MAC: How to install flutter on MAC

Introduction To ANDROID STUDIO

Dart programming basics

- Variables
- Data types
- Arithmetic operators
- Dot operator
- Collection data
- List
- Set
- Maps
- Comparison operators
- Conditional statements
- Logical operator
- Null safety
- Loops

*Functions Overview

Variables

variables are used to store values of different types

Data type / variable = value;

Types of Data

1- Primitive Data type: with single

- Int - Double- bool – String – Dynamic – var

2- Collection Data type: with multiple values

- List – Set - Maps

Primitive Data Types

- **int:** Used to store integer values (whole numbers without decimal points).
  Example: int age = 25;
- **double**: Used to store floating-point values (numbers with decimal points).
  Example: double pi = 3.14;
- **bool:** Used to store boolean values (true or false).
  Example: bool isRaining = true;
- **String**: Used to store sequences of characters or text.
  Example: String name = "John Doe";
- **dynamic**: Used to store values of any type. The type can be changed during runtime.
  Example: dynamic variable = 10;
- **var**: Used to declare a variable without specifying its type explicitly. The type is inferred from the assigned value.
  Example: var count = 5;

Arithmetic operators

- Addition (+): Adds two operands together.
  Example: int sum = 5 + 3; // sum will be 8

- Subtraction (-): Subtracts the second operand from the first operand.
  Example: int difference = 10 - 5; // difference will be 5

- Multiplication (*): Multiplies two operands together.
  Example: int product = 4 * 6; // product will be 24

- Division (/): Divides the first operand by the second operand. The result will be a double value.
  Example: double quotient = 10 / 3; // quotient will be approximately 3.3333

- Integer Division (~/): Divides the first operand by the second operand and returns the integer result by truncating any decimal places.
  Example: int quotient = 10 ~/ 3; // quotient will be 3

- Remainder/Modulus (%): Returns the remainder of the division of the first operand by the second operand.
  Example: int remainder = 10 % 3; // remainder will be 1

- Increment (++): Increases the value of a variable by 1.
  Example: int number = 5; number++; // number will be 6

- Decrement (--): Decreases the value of a variable by 1.
  Example: int number = 5; number--; // number will be 4

TASK 1

- Create a Dart program that declares a variable called "age" and assigns it the value of your age. - Declare a variable called "name" and assign it your name. - Print out the following sentence using the variables you declared: "My name is [name] and I am [age] years old."

- Perform the following arithmetic operations:
  a. Declare two variables, "num1" and "num2", and assign them any numerical values.
  b. Add "num1" and "num2" and store the result in a new variable called "sum".
  c. Subtract "num2" from "num1" and store the result in a new variable called "difference".
  d. Multiply "num1" and "num2" and store the result in a new variable called "product".
  e. Divide "num1" by "num2" and store the result in a new variable called "quotient".

Collection Data Type

- List: represents an ordered collection of objects. It allows duplicate values and provides methods to add, remove, and access elements.
- Set: represents an unordered collection of unique objects. It doesn't allow duplicate values and provides methods for set operations like union, intersection, and difference.
- Map: represents a collection of key-value pairs. It associates keys with values, and each key is unique within the map.

List

- List<Data type> List-Name = [Value1,Value2,Value3,…..]; - square brackets [];
- Add value to List:
- List-Name.add(value);
- Remove Value from List:

List-Name.remove(value);

Set

- Syntax: Set<Data type> Set-Name = {Value}; - curly brackets {Value1,Value2,Value3,……};
- Add value to Set:

Set-Name.add(value);

- Remove Value from Set:

Remove value from Set:

Set-Name.remove(value);

Maps

- Syntax:

Map<data type of unique key, date type of value> Map-Name = {key1:value1,key2:value2,key3:value3,….}

- Add value to Map:

Map-Name[key] = value;

Remove value from Map:

Map-Name.remove(key);

Dot operator

- The dot operator (.) can also be used with certain objects or types to access predefined properties or methods that are available for those objects or types. These properties and methods are part of the Dart language or standard libraries and are already imported.

Comparison operators

- Equality (==): Checks if two values are equal. (=)
  Example: 5 == 5 (returns true)

- Inequality (!=): Checks if two values are not equal. (≠)
  Example: 5 != 3 (returns true)

- Greater than (>): Checks if the left operand is greater than the right operand.
  Example: 5 > 3 (returns true)

- Less than (<): Checks if the left operand is less than the right operand.
  Example: 5 < 3 (returns false)

- Greater than or equal to (>=): Checks if the left operand is greater than or equal to the right operand. (≥)
  Example: 5 >= 5 (returns true)

- Less than or equal to (<=): Checks if the left operand is less than or equal to the right operand. (≤)
  Example: 5 <= 3 (returns false)

Task 2

- Create a list called "fruits" and add the following fruits to it: "apple", "banana", "orange". - Print out the second fruit in the list.
- Create a set called "countries" and add the following countries to it: "USA", "Canada", "Mexico". - print out the result.
- Create a map called "person" and add the following key-value pairs to it:
  a. Key: "name", Value: your name.
  b. Key: "age", Value: your age.
  c. Key: "country", Value: your country of residence.
- -Print out the value associated with the key "age" in the "person" map.
- Create a List of int and print using comparison operators (=,<,>,!= ,>=,<=)
- Create a set of double and print using comparison operators (=,<,>,!= ,>=,<=)
- Create a Map key (int) value (dynamic) and print using comparison operators (=,<,>,!= ,>=,<=)

Conditional statements

- If statement:
  The if statement evaluates a condition and executes a block of code if the condition is true. It can also be followed by an optional else clause to execute a different block of code when the condition is false.

```
if (condition) {
  // Code to execute if the condition is true
} else {
  // Code to execute if the condition is false
}
```

- If- else if - else : (Nested if)
  Allows you to check multiple conditions and execute different blocks of code based on the first condition that evaluates to true.

```
if (condition1) {
  // Code to execute if condition1 is true
} else if (condition2) {
  // Code to execute if condition2 is true
} else {
  // Code to execute if all conditions are false
}
```

Logical operator

logical operations on boolean values or expressions. Dart provides three logical operators: && (logical AND), || (logical OR), and ! (logical NOT).

- Logical AND (&&): The && operator returns true if both the left and right operands are true, otherwise it returns false. It evaluates the right operand only if the left operand is true.
- Logical OR (||): The || operator returns true if either the left or right operand is true, or if both are true. It evaluates the right operand only if the left operand is false.
- Logical NOT (!): The ! operator negates the boolean value of its operand. If the operand is true, ! returns false, and if the operand is false, ! returns true.

Conditional statements

- Switch statement:
  The switch statement provides a way to choose different blocks of code to execute based on the value of an expression. It is often used when you have multiple possible values and want to perform different actions for each value.

```
switch (expression) {
  case value1:
    // Code to execute if expression matches value1
    break;
  case value2:
    // Code to execute if expression matches value2
    break;
  // ...
  default:
    // Code to execute if expression doesn't match any case
}
```

Loops

- for Loop:
- The for loop is used to iterate over a sequence of values, such as a list or a range of numbers.

Syntax :

```
for (initialization; condition; increment/decrement) {
  // Code to be executed in each iteration
}
```

Loops

- for-in Loop:
  The for-in loop iterates over each element of an iterable (such as a list, set, or map) and performs a specified action for each element.

```
for (var element in iterable) {
  // Code to be executed for each element
}
```

- forEach Loop:
  The forEach loop is a higher-order function available on iterables (such as lists) that allows you to iterate over each element and execute a specified function for each element.

```
iterable.forEach((element) {
  // Code to be executed for each element
});
```

Loops

- while Loop:
- The while loop is used to repeatedly execute a block of code as long as a given condition is true.

Syntax :

```
while (condition) {
  // Code to be executed in each iteration
}
```

Loops

- Do – While :
- It is a type of loop that executes a block of code repeatedly as long as a specified condition is true. The key difference between the do-while loop and other loop structures like for and while is that the do-while loop guarantees that the block of code is executed at least once, regardless of whether the condition is initially true or false.

Syntax :

```
do {
  // Code to be executed
} while (condition);
```

Task 3

- 1- Write a program to assign letter grades based on a student's score.
- Ex : if score from 90-100 print A – if score from 80 – 89 print B – if score from 70 – 79 print c and so on
- Write 2 programs one by if using nested if and other using switch – case
- 2- Check if a given year is a leap year or not. (logical operator)
- A year is a leap year if: it is divisible by **4**. It is **not divisible by 100**, unless it is divisible by **400**.
- 3- Use a **for loop** to display the multiplication table of a given number.
- 4- Calculate the sum of all elements in a list using a **for-in loop**.
- 5- Write a program that repeatedly asks the user to guess a number until they get it right. (while - do)
- If the guess is correct, print "Congratulations! You guessed it." - If the guess is wrong, print "Try again!"
- 6- Use a **do-while loop** to repeatedly show a menu for basic math operations.
- 1. Addition 2. Subtraction 3. Exit - If the user selects **1** or **2**, ask for two numbers and perform the operation. If the user selects **3**, exit the program.

Functions overview

- Basic Function
- Return Statements
- Parameters
- Parameters and arguments
- Optional Parameters
- Named Parameters

Basic Function

- blocks of code that take input and return a result to perform a specified action. The function is a piece of code that we can reuse instead of writing it multiple times. Functions allow programmers to break a problem into smaller parts, each of which performs a specific task. The main aim of using the function is the **DRY (Don’t Repeat Yourself)** concept.

```
void functionName ()
{
 //Body
}
```

Return type Functions

- String , int , double, Map, List , Set.
- Return statement should be with the same type .

```
returnType functionName() {
  // Function body
  // Statements and logic
  return value; // Return the value
}
```

Parameters

- Parameters is used to path variables to make an operation and return its value after the operation done

```
returnType functionName(parameters) {
  // Function body
  // Statements and logic
  return value; // Optional
}
```

Parameters & Arguments

- Parameters is the define of the variables that will use inside the functions.
- Arguments is the value that using while calling the functions

```
int add(int a, int b) {
  int sum = a + b;
  return sum;
}
```

```
int result = add(3, 4); // result will be 7
```

Optional Parameters

- Optional positional parameters are defined using square brackets [ ] around the parameter names. These parameters are optional, and the function can be called without providing values for them.

```
String greet(String name, [String? title]) {
  if (title != null) {
    return "Hello, $title $name!";
  } else {
    return "Hello, $name!";
  }
}

// Calling the function
print(greet("Alice")); // Output: Hello, Alice!
print(greet("Alice", "Dr.")); // Output: Hello, Dr. Alice!
```

Named Parameters

- Named parameters are defined using curly braces {} around the parameter names. These parameters are also optional, and the function can be called without providing values for them. Named parameters are identified by their names when calling the function. named parameters give parameters to be optional , if you need to be required you can use required key word.

```
String greet({required String name, String? title}) {
  if (title != null) {
    return "Hello, $title $name!";
  } else {
    return "Hello, $name!";
  }
}

// Calling the function
print(greet(name: "Alice")); // Output: Hello, Alice!
print(greet(name: "Alice", title: "Dr.")); // Output: Hello, Dr. Alice!
```

Scoop

In Dart, scope refers to the visibility and accessibility of variables, functions, and classes within different parts of your code. Dart has several levels of scope, including global scope, class scope, and block scope.

1. Global Scope:
   Variables and functions defined outside of any class or function have global scope. They can be accessed from anywhere within the Dart file.

2. Block Scope:
   Block scope refers to variables defined within blocks of code, such as within loops, conditional statements, or function bodies. Variables defined within a block are only accessible within that block.

3. Class Scope:
   Variables and functions defined within a class have class scope. They can be accessed within the class using the this keyword or without it, depending on the context.

Null safety

Null safety in Dart offers a more reliable and expressive type system that helps catch null-related errors early during development, improving code quality and reducing runtime crashes caused by null references.

1. Nullable and Non-nullable Types:
   In Dart's null safety, variables are classified into two types: nullable and non-nullable. Nullable types: By default, variables can hold null values if their type is not explicitly marked as non-nullable. Nullable types are denoted with a ? Suffix.

2. Late Variables:
   Dart introduces the late keyword to indicate variables that are guaranteed to be assigned a non-null value before they are accessed, even if they don't have an initial value. This allows you to delay the initialization of non-nullable variables.

3. Null Assertion Operator (!):
   To explicitly assert that a nullable variable is indeed non-null at a given point, you can use the null assertion operator (!). It tells the analyzer that you are certain the value is not null, even if the type system cannot prove it.

4. The ?? operator is known as the null-aware coalescing operator, and it is used to provide a default value when encountering a null value. It is a concise way to handle null values and avoid null reference errors.

C2 General

Task 4

Let's create a program that calculates the area of different shapes (square, rectangle, and circle) using functions In this program, we have defined three functions:

- calculateSquareArea(): This function takes the side length of a square as a parameter and calculates its area.
- calculateRectangleArea(): This function takes the length and width of a rectangle as parameters and calculates its area.
- calculateCircleArea(): This function takes the radius of a circle as a parameter and calculates its area.
- In the main() function, we call each of these functions with the appropriate parameters, store the returned values in variables, and then print the calculated areas.